Zephyr Project

IT-Schulungen.com
Technology & Transformation

Devicetree in native_sim

Erklärung am generierten Baum

1.12

Vom Hardwareknoten zum C-Makro

Fünf Schritte durch den Devicetree-Pfad

01

Beschreiben

Knoten und Properties bilden Hardwarestrukturen ab

02

Zusammensetzen

Board-DTS, Includes und Overlays werden verbunden

03

Validieren

Bindings und compatible definieren Bedeutung und Typen

04

Generieren

Der finale Baum erzeugt Header und C-Makros

05

Verwenden

Code greift über DT_PATH, DT_NODELABEL und DT_PROP zu

Ergebnis: Ein Knoten wird gefunden, eine Property gelesen und bis zum C-Makro verfolgt.

Drei Ebenen klar trennen

DEVICETREE

Hardwarebeschreibung

Geräte und Instanzen

Adressen und Interrupts

Pins und Properties

Verbindungen und Status

KCONFIG

Softwareauswahl

Treiber aktivieren

Subsysteme konfigurieren

Features auswählen

Build-Inhalte steuern

LAUFZEIT

Programmparameter

Simulatoroptionen

Dateipfade und Ports

Testeingaben

Prozessumgebung

Devicetree beschreibt Struktur – Kconfig aktiviert Software – Laufzeitparameter steuern den Prozess.

Ein Knoten besteht aus Namen und Properties

TRAININGSKNOTEN

```
training: training@1000 {
  compatible = "training,demo";
  reg = <0x1000 0x100>;
  status = "okay";
  interval-ms = <1000>;
};
```

01 Knotenidentität

Label, Knotenname und Unit-Adresse dienen unterschiedlichen Referenzwegen.

02 Properties

compatible, reg, status und eigene Werte beschreiben den Knoten.

Die Unit-Adresse folgt normalerweise der ersten Adresse in reg; das Label ist eine Quelltextreferenz.

Der finale Tree entsteht in Schichten

ZUSAMMENSETZUNG

```
SoC- und Architektur-Includes
            +
Board-DTS und gemeinsame Includes
            +
Anwendungs- und Board-Overlays
            ↓
build/zephyr/zephyr.dts
```

01 Basis

SoC und Board liefern die vorhandene Hardwarestruktur.

02 Anpassung

Overlays ändern oder ergänzen ausgewählte Knoten.

03 Ergebnis

zephyr.dts zeigt den vollständig zusammengeführten Baum.

Nicht die Quelldatei erraten: Für die wirksame Sicht immer build/zephyr/zephyr.dts prüfen.

Referenzen und Status verstehen

`Label`	verweist im DTS-Quelltext mit &label auf einen Knoten
`Alias und chosen`	geben Rollen wie console oder application-spezifische Kurzreferenzen an
`Phandle`	verknüpft Knoten über Properties, etwa Controller und Verbraucher
`status`	okay aktiviert den Knoten; disabled verhindert seine Verwendung
`Pfad`	adressiert einen Knoten eindeutig über seine Position im Baum

Label, Alias und Pfad sind Referenzwege; status entscheidet, ob ein Knoten verfügbar ist.

Bindings geben Properties Bedeutung

COMPATIBLE VERBINDET KNOTEN UND BINDING

Ein Binding beschreibt erlaubte Properties, Typen, Pflichtfelder und die Beziehung zum Treiber.

`compatible`	wählt das passende Binding für einen Knotentyp aus
`Property-Typ`	legt etwa string, int, array, phandle oder phandle-array fest
`required`	macht eine Property für diesen kompatiblen Knotentyp verpflichtend
`Treiberbezug`	C-Code kann Instanzen eines compatible gezielt verarbeiten
`Diagnose`	Binding-Fehler weisen auf unbekannte oder falsch typisierte Properties hin

compatible ist keine freie Beschreibung: Der String muss zum vorgesehenen Binding passen.

Knoten finden und aus C lesen

DEVICETREE UND C

```
$ grep -n 'training@1000' build/zephyr/zephyr.dts


#define NODE DT_NODELABEL(training)
BUILD_ASSERT(DT_NODE_HAS_STATUS(NODE, okay));
uint32_t ms = DT_PROP(NODE, interval_ms);
```

01 Knoten-ID

DT_NODELABEL, DT_PATH oder ein Alias liefert eine compile-time Knotenkennung.

02 Property

DT_PROP liest den typisierten Wert; Bindings steuern die Generierung.

Bindestriche in Property-Namen erscheinen in Makroargumenten als Unterstriche: interval-ms → interval_ms.

Devicetree verstanden

ERFOLG

Pfad nachvollzogen

Knoten gefunden
Property gelesen
Binding zugeordnet
C-Makro erklärt

NATIVE_SIM

Modell statt MCU

simulierte Peripherie
Host-Schnittstellen
keine konkrete MCU
elektrisches Verhalten offen

NÄCHSTER SCHRITT

Praxis 1.13

Overlay anlegen
Property verändern
Build prüfen
Fehler diagnostizieren

Lernziel: Einen Knoten finden, eine Property lesen und den Weg zum C-Makro verstehen.